# DELTA_ANALYSIS_20260712

# Helix OTA — NET-NEW Delta Analysis (server + submodules/ota-*)

**Revision:** 1 **Last modified:** 2026-07-12T00:00:00Z **Authority:** §11.4.172 (living production-readiness planning) · §11.4.93 (workable-items DB load-list) · §11.4.6 (no-guessing — every claim cites a path; nothing runtime-reproduced this pass) · §11.4.165 (findings cross-checked by an independent verification pass) **Scope:** the Go control plane `server/` and the six `submodules/ota-*` bricks (ota-protocol, ota-telemetry-schema, ota-artifact-validator, ota-rollout-engine, ota-update-engine-bridge, ota-android-agent). **Method:** READ-ONLY static read of source + git history + `docs/workable_items.db`, de-duplicated against the 40 tracked items OTA-022..061 in `docs/planning/PRODUCTION_READINESS_PLAN.md`. No builds/tests were run; no code was modified. Every item below is a genuine gap NOT already a tracked workable-item.

---

## 1. Executive delta summary

This document records ONLY the DELTA on top of the already-tracked set (OTA-003..061). It adds 7 **proposed NET-NEW workable-items** (SRV-NEW-1..4 server, SUB-NEW-1..2 submodules) and **5 untracked known issues** (KI-1..KI-5). The two most material findings: **KI-1** — OTA-033 (SEC-1 token-secret fail-fast) is already committed and clean in the tree, yet it is still tracked `Queued` and the plan's Top-Risk #1 still reads as open; and **SRV-NEW-1** — the server has no versioned DB-migration framework, a silent prerequisite the Accounts M1 stream (OTA-046) assumes exists.

Baseline facts that produced NO net-new item (recorded so the conductor does not re-chase them): every `server/` package directory has ≥1 `_test.go`; both store backends (`MemoryRepository`, `PostgresRepository`) satisfy the full `store.Repository` interface with method parity; every HTTP route is behind `authMiddleware+requireRole` (no unauthenticated mutating route); `transport.Server.Shutdown(ctx)` already exists (`server/internal/transport/transport.go:150`), so OTA-032's residual work is only the SIGTERM signal-wiring in `main.go` for both the plain-HTTP and QUIC paths; and the `android/build/` trees in the two Kotlin bricks are NOT git-tracked (checked — no §11.4.30 versioned-artifact violation).

---

# 2. NET-NEW items

### [SRV-NEW-1] Type: Task · Severity: MED · Phase: P1

**Title:** No versioned DB-migration framework; schema is one idempotent DDL blob re-run on boot.

**Description (§11.4.171):** The Postgres store brings up its schema by `//go:embed`-ing a single `internal/store/schema_postgres.sql` (272 lines: 18 `CREATE TABLE IF NOT EXISTS` + 3 `ALTER TABLE` + 13 `CREATE INDEX IF NOT EXISTS`) and executing the whole file on every boot via `PostgresRepository.Migrate` (`server/internal/store/postgres.go:49-56`); the rollout store does the same with `rollout_schema.sql` (`server/internal/rollout/postgres.go:45`). There is no `schema_migrations`/applied-version ledger, no `goose`/`golang-migrate` tooling, no ordered/numbered forward-migration files, and no down/rollback path — schema evolution across releases is ad-hoc and unauditable, and altering an existing table on a populated production database is not reliably expressible in a wholesale idempotent re-exec. It matters because a fleet control-plane changes its schema many times over its life and needs safe, recorded, forward-only migrations with a version ledger. Verify by reading `Migrate()` and the two `.sql` files (there is no version table and no migrations directory beyond the two blobs). Operators doing safe rolling schema upgrades and future authors both benefit; the expected outcome is a real migration runner plus numbered migration files. This is also the clean prerequisite the Accounts plan already assumes when it references "002/003 migrations" — but the base ships no `001` framework to build on.

**Already-covered-by:** none (OTA-046 Accounts M1 assumes numbered migrations exist; it does not create the framework).

## [SRV-NEW-2] Type: Bug · Severity: MED · Phase: P1

**Title:** `/readyz` never fails — the readiness probe ignores the DB result and always returns true.

**Description (§11.4.171):** The `/readyz` route is wired to `health.Checker.Ready` (`server/internal/api/server.go:174`), which delegates to the `ReadyFunc` built in `main.go`. That closure calls `repo.GetIdempotent(ctx, "__readyz__")`, logs, and then `return true` **unconditionally**, ignoring the repository result (`server/cmd/ota-server/main.go:86-93`); its own comment admits it is "a liveness stand-in for the real PostgreSQL/MinIO probes used in production." So a Postgres or object-storage outage never flips readiness to NOT-READY, and a Kubernetes / load-balancer health gate keeps routing traffic to a pod that cannot serve. It matters because a readiness signal that can never go not-ready defeats rolling-deploy draining and outage isolation. Verify by reading the ReadyFunc closure — no branch returns false. Operators and SREs benefit; the expected outcome is a readiness probe that actually pings the pool (and object storage once it lands) and returns false on failure.

**Already-covered-by:** none (OTA-034 observability is metrics / structured-logs / tracing, not readiness correctness).

## [SRV-NEW-3] Type: Task · Severity: LOW · Phase: P1

**Title:** Project-member management is unwired — `ListProjectMembers`/`RemoveProjectAccess` implemented but no HTTP route.

**Description (§11.4.171):** The `store.Repository` interface declares `ListProjectMembers`, `RemoveProjectAccess`, `SetProjectAccess`, and `GetProjectAccess`, and both backends implement all four (memory + pgx, full parity), but the only mounted project routes are CRUD — `POST/GET/GET/PATCH/DELETE /projects[/:projectId]` (`server/internal/api/server.go:246-250`); there is no `GET/POST/DELETE /projects/:projectId/members` route and no member handler in `handlers_project.go` (only Create/List/Get/Update/Delete project). `SetProjectAccess` is called only to seed the creator's admin ACL, and `ListProjectMembers`/`RemoveProjectAccess` are referenced only by tests — they are unwired production methods. It matters per §11.4.124 (investigate-before-remove / finish-the-wiring): either the member-management endpoints

should be mounted, or the methods explained and kept as a deliberate seam. Verify with a route grep (no `members` under `/projects`) and a caller grep (only tests + the store). Admins who need to grant/revoke project access benefit; the expected outcome is mounted, role-gated member routes or a documented keep-as-seam decision.

**Already-covered-by:** partial — the Accounts stream (OTA-048 M3 / OTA-049 M4) rescopes project/account authz, but no tracked item wires these existing pre-Accounts project-member methods, and they are not OTA data routes.

## [SRV-NEW-4] Type: Bug · Severity: LOW · Phase: P1

**Title:** One-of-pair TLS config silently downgrades to plain HTTP.

**Description (§11.4.171):** `main.go` enters the HTTP/3 + HTTP/2 TLS path only when BOTH `HELIX_TLS_CERT` and `HELIX_TLS_KEY` are set (`server/cmd/ota-server/main.go:119`, an `&&` guard); if an operator sets exactly one of the pair, the server silently serves plain HTTP on the plaintext port with no error and no warning, and `config.Load()` performs no cross-field validation of the pair (`server/internal/config/config.go:132-135`). It matters because a half-configured TLS deployment looks "up" while actually terminating no TLS — a security-relevant silent misconfiguration that a deployment can carry into production undetected. Verify by reading the `&&` guard in `main.go` and the absence of any TLS-pair check in `Load()`. Operators benefit from fail-fast behavior; the expected outcome is `Load()` erroring (or main loudly warning) when exactly one of the pair is set. Good fold-in candidate for a broader config-validation hardening pass.

**Already-covered-by:** none.

## [SUB-NEW-1] Type: Task · Severity: MED · Phase: P0/P5

**Title:** All six ota-* bricks lack the mandated §11.4.31 `helix-deps.yaml` (and ship no `upstreams/` recipes).

**Description (§11.4.171):** None of `ota-protocol`, `ota-telemetry-schema`, `ota-artifact-validator`, `ota-rollout-engine`, `ota-update-engine-bridge`, `ota-android-agent` contains a `helix-deps.yaml` dependency manifest, and none has an `upstreams/` (or legacy `Upstreams/`) recipe directory — confirmed present-vs-absent by contrast, since the other owned submodules (`containers`, `http3`, `security`, `doc_processor`) all DO ship `helix-deps.yaml`, and the two Go bricks spot-checked have a single `origin` push URL rather than the §2.1 four-mirror fan-out. It matters

because §11.4.31 makes the manifest the machine-readable dependency-graph bridge a consumer uses to reconstruct the graph, and §11.4.36/§2.1 make `upstreams/` the mechanism each brick uses to reach all four mirrors. Verify with `ls submodules/ota-*/helix-deps.yaml` (none) and `ls -d submodules/ota-*/upstreams` (none), contrasted with the other owned submodules (present). Downstream consumers and multi-mirror release hygiene benefit; the expected outcome is a `helix-deps.yaml` per brick plus `upstreams/` recipes (or an explicit operator decision that these bricks are single-mirror).

**Already-covered-by:** none (the plan covers the llm*_/vision_engine mirror-forks under OTA-026 and containers hardening under OTA-058, but not ota-* manifests/upstreams).

## [SUB-NEW-2] Type: Task · Severity: LOW · Phase: P1

**Title:** No wire/telemetry message-schema-version field for cross-version device↔server evolution.

**Description (§11.4.171):** The `ota-protocol` request/offer/telemetry payload types carry no `ProtocolVersion`/`SchemaVersion`/`MessageVersion` field (the `Version`/`CurrentVersion`/`MinCurrentVersion` fields are firmware/OS versions, not message-schema versions — `submodules/ota-protocol/types.go`), and `ota-telemetry-schema` has no `version` token at all across its three sources (`codec.go`, `event.go`, `health.go`). The only versioning present is the coarse REST URL path `/api/v1` (`config.DefaultAPIBasePath`). It matters because an OTA fleet spans many device-agent versions while the control plane keeps evolving; without an embedded per-message schema version there is no compatibility negotiation, so a payload-schema change can silently break older or newer peers. Verify by grepping both modules for a schema/protocol version field (absent). Long-lived heterogeneous fleets and safe schema evolution benefit; the expected outcome is an explicit schema-version field on the wire messages plus tolerant decoding. Honest boundary: `/api/v1` gives coarse REST versioning, so this is a graceful-evolution gap, not a total absence of versioning.

**Already-covered-by:** none.

---

# 3. Known issues (untracked)

## KI-1 — OTA-033 (SEC-1 token-secret fail-fast) is already DONE but tracked "Queued", and the plan's Top-Risk #1 reads as open.

**Fact:** SEC-1 fail-fast on unset `HELIX_TOKEN_SECRET` is COMMITTED at `7763c7c7` and the working tree is clean; `server/internal/config/config.go:179-207` now REFUSES to boot on an unset secret (dev fallback gated behind `HELIX_ALLOW_INSECURE_DEV_TOKEN_SECRET` with a loud warning). Yet DB item `OTA-033` status = `Queued`, and `docs/planning/PRODUCTION_READINESS_PLAN.md:17` (Top-5 Risk #1) plus §2.7 still describe the *silently-defaulting* dev secret citing `config.go:180-184`. This is §11.4.172/§11.4.93 living-plan / DB drift. **Evidence:** `git show 7763c7c7`; `git status --porcelain server/internal/config/config.go` (clean); `git show HEAD:server/internal/config/config.go | grep "refusing to start"` (present); `docs/workable_items.db` OTA-033 = Queued. **Recommended action:** close OTA-033 with the correct §11.4.33 vocabulary and correct the plan's Risk #1 + §2.7 to reflect that SEC-1 has landed.

## KI-2 — `workable-items` tooling is a divergent simplified reimplementation whose `db-to-md` sync is an explicit stub.

**Fact:** The in-repo `cmd/workable-items/main.go` is a single-file simplified reimplementation whose bidirectional sync is a stub — `main.go:546` ("Sync: MD <-> DB (bidirectional stub / framework)") and `main.go:937-938` (prints `WARN: db-to-md sync is a stub`), so the §11.4.93 byte-identical round-trip cannot regenerate the docs from the DB. The canonical richer engine lives at `constitution/scripts/workable-items/cmd/workable-items/` (ships `diary_cmd.go`, `obsolete.go`, `assign_test.go`, more) and per §11.4.93/§11.4.106/§11.4.177 should be consumed BY REFERENCE rather than reimplemented in-repo. **Evidence:** `cmd/workable-items/main.go:546,937-938`; contrast with `constitution/scripts/workable-items/cmd/workable-items/*`. **Recommended action:** adopt the constitution engine by reference (retire the local simplified copy) so `db-to-md` and the full column set work.

## KI-3 — `workable_items.db` schema is missing constitution-mandated columns/tables.

**Fact:** `PRAGMA table_info(items)` returns only `ota_id, type, status, severity, title, description, composes_with, current_location, created_at, modified_at`. Missing: `created_by`/`assigned_to` (§11.4.104); a `canonical_track` column + authoritative destination-branch binding and a `logic_groups` table (§11.4.181/§11.4.191 — no `logic_groups` table exists); a `test_diary` table (§11.4.149); a `reopens_count` column (§11.4.55). `.tables` shows only `items`, `item_history`, `meta`. (The `OTA-NNN` id prefix vs the §11.4.54 `ATM-NNN` convention is a project-specific naming choice and is NOT counted as a defect here.) **Evidence:** `sqlite3 docs/workable_items.db "PRAGMA table_info(items)"`; `sqlite3 docs/workable_items.db ".tables"`. **Recommended action:** extend the schema with the §11.4.104 / §11.4.191 / §11.4.149 / §11.4.55 columns + tables (naturally landed together with KI-2's engine adoption).

## KI-4 — 12 closed items violate §11.4.33 type-aware closure vocabulary.

**Fact:** Exactly 12 closed items are recorded as `Fixed (→ Fixed.md)` regardless of Type — 7 Features (should be `Implemented (→ Fixed.md)`: OTA-005, 006, 008, 015, 016, 017, 018) and 5 Tasks (should be `Completed (→ Fixed.md)`: OTA-007, 009, 010, 014, 019); only the 1 Bug (OTA-020) is correct. A `workable-items validate` run would FAIL on these. **Evidence:** `sqlite3 docs/workable_items.db "SELECT type, count(*) FROM items WHERE status='Fixed (→ Fixed.md)' GROUP BY type"` → Bug 1, Feature 7, Task 5. **Recommended action:** retype-close the 12 rows to the type-appropriate terminal status per §11.4.33.

## KI-5 — Correction to OTA-027 / plan-K2 framing: `submodules/LLMProvider` is a symlink, not a stray directory.

**Fact:** `submodules/LLMProvider` is a symbolic link to `llm_provider` (`ls -la submodules/` shows `LLMProvider -> llm_provider`), NOT an untracked duplicate directory copy as OTA-027 ("remove stray untracked submodules/LLMProvider dir") and plan-K2 frame it. **Evidence:** `ls -la submodules/` (symlink entry). **Recommended action:** re-verify OTA-027 as a symlink-removal (and confirm nothing resolves the `LLMProvider` path) before acting, rather than a directory-copy removal.

---

# 4. Untested surfaces (file → what's untested)

Server package coverage is genuinely strong — **every** `server/` package directory (including `cmd/*`, `tools/loadtest`, `tests/chaos`, `tests/stress`) has at least one `_test.go`, and the `store.Repository` contract is asserted on both backends. There is **no untested package** in `server/`. The residual per-file / per-type gaps below sit largely inside the already-tracked §11.4.169 item (OTA-040) and the Android item (OTA-043); they are enumerated here as the concrete surfaces:

- `submodules/ota-update-engine-bridge/android/src/main/.../BootStateObserver.kt` — the concrete `SystemPropertyBootStateObserver` class has real logic but no host-runnable unit test; the module has no `src/androidTest/`. JVM unit only; no stress/chaos/bench/memory. (→ overlaps OTA-043; the host-runnable JVM/Robolectric unit gap is not explicitly enumerated there.)
- `submodules/ota-android-agent/android/src/main/.../apply/ReflectiveUpdateEngineApplyPort.kt` — concrete reflection apply-port class, no host-runnable unit test.
- `submodules/ota-android-agent/android/src/main/.../poll/PollScheduler.kt` — `object` with WorkManager `schedule()` logic, no unit test.
- `submodules/ota-android-agent/core/src/main/.../protocol/Dtos.kt` + `protocol/Enums.kt` (`fromWire` logic) — no dedicated test file; exercised only indirectly via `CodecRoundTripTest`.
- The two Kotlin bricks' `androidTest/` (`OtaAgentOnDeviceTest.kt`) cannot run in a host build (needs a system-UID / platform-signed target — noted in the module `BUILD_STATUS.md`). (→ OTA-043 real-device coverage.)
- 4 Go bricks (`ota-protocol`, `ota-telemetry-schema`, `ota-artifact-validator`, `ota-rollout-engine`) — heavy unit/fuzz coverage (tests > sources) but zero `Benchmark` funcs and no memory/leak tests. (→ OTA-040.)
- `server/internal/device/applyport.go` — SCAFFOLD; its runtime apply loop is untested because unimplemented. (→ OTA-038.)

---

# 5. Honest coverage gaps (subsystems not fully inspected)

- **No build/test executed (READ-ONLY session).** "Strong test coverage" is asserted from file presence + git history, not re-run this

pass; the migration, `/readyz`, TLS-pair, and schema-version findings are static-read facts, not runtime-reproduced.

- **Web frontends** (`clients/ota-manager/`, `dashboard/`) were out of the requested scope (server + ota-*); the plan already tracks their stress/chaos/DDoS/memory/host-render gaps (§2.2/§3, OTA-055..057).
- **Non-ota owned submodules not independently surveyed this pass:** `challenges`, `containers`, `doc_processor`, `helixqa`, `http3`, `llm_orchestrator`, `llm_provider`, `llms_verifier`, `security`, `vision_engine`. They were checked only for `helix-deps.yaml` presence (all present). Their known items are covered by the plan (mirror-forks OTA-026/K1; containers + vm/emu hardening OTA-058/059); their other internal open items were out of this task's server+ota-* scope.
- **Multi-mirror check for SUB-NEW-1** was `git remote -v` spot-verified on 2 of 6 bricks; the `helix-deps.yaml`/`upstreams/` absence is confirmed on all 6.
- **Runtime/on-device behavior** (Android A/B apply, RK3588 silicon) is unexercised in-repo by design and already tracked (OTA-003/004/042/043/044).

---

*Prepared as a READ-ONLY analysis + planning deliverable. No fixes were implemented, no DB was written, no git operations were performed. Every claim cites a file/path or a reproducible query. Effort/phase tags are advisory, aligned to the phases in* `docs/planning/PRODUCTION_READINESS_PLAN.md`.